

ARCH TECHNOLOGIES

INTERNSHIP PROJECT REPORT

MNIST Digit Recognition Using Machine Learning

Full Name: Karan Kumar

Internship Domain: Machine Learning

Email: karankhatri952@gmail.com

Phone: 03312428152

Submission Month: Month 1

1. Abstract

This project focuses on developing an **MNIST Digit Recognition system using Machine Learning**. The objective is to build a model that can recognize and classify handwritten digit images from **0 to 9**.

The MNIST dataset contains grayscale images of handwritten digits. Each image has a size of **28 × 28 pixels**, resulting in **784 pixel features**. The images were pre-processed by separating the target labels from pixel values and normalizing the pixel values.

A **K-Nearest Neighbors (KNN)** classification algorithm was trained using the processed image data. The trained model was then used to predict digits from the test dataset. Model performance was evaluated using **accuracy, classification report, and confusion matrix**.

The project demonstrates how Machine Learning can be applied to image classification and handwritten digit recognition.

2. Introduction

Handwritten digit recognition is a common Machine Learning and image classification problem. Computers cannot directly understand an image as a digit, so image pixels must first be converted into numerical features that a Machine Learning algorithm can process.

In this project, the MNIST dataset is used to train a Machine Learning model capable of identifying handwritten digits from **0 to 9**.

The project follows a complete Machine Learning workflow, including data loading, preprocessing, visualization, model training, prediction, and evaluation.

3. Problem Statement

The objective of this project is to develop a Machine Learning model that can automatically recognize handwritten digit images and classify them into one of the ten classes:

- 0
- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9

4. Objectives

- Load and analyze the MNIST dataset.
- Understand the structure of handwritten digit images.
- Pre-process the image data.
- Separate image features and target labels.
- Normalize pixel values.
- Visualize handwritten digit images.
- Train a Machine Learning classification model.
- Predict handwritten digits from test data.
- Evaluate model accuracy.
- Generate a confusion matrix and classification report.
- Analyze the model's predictions.

5. Tools and Technologies

- Python
 - Jupyter Notebook
 - Pandas
 - NumPy
 - Matplotlib
 - Seaborn
 - Scikit-learn
 - K-Nearest Neighbors (KNN)
-

6. Dataset

The project uses the **MNIST handwritten digit dataset** obtained in CSV format.

The dataset contains handwritten digit images represented by pixel values.

Each image has:

- **28 × 28 pixels**
- **784 pixel features**
- One **label** representing the actual digit

The label contains one of the ten classes from **0 to 9**.

The dataset used in this project contains **10,000 records** and **785 columns**, including the label column and 784 pixel columns.

7. Data Preprocessing

Data preprocessing was performed before training the Machine Learning model.

The following steps were completed:

Separation of Features and Label

The label column was separated from the pixel data.

- **X:** Pixel values of the images
- **y:** Actual digit labels

Pixel Normalization

The original pixel values range from **0 to 255**.

The values were normalized by dividing them by 255 so that they fall within the range **0 to 1**.

This helps the Machine Learning model work with standardized numerical values.

Image Representation

Each handwritten digit is represented as a **28 × 28 pixel image**.

For Machine Learning, these pixels are treated as **784 numerical features**.

8. Data Visualization

Sample images from the dataset were displayed using Matplotlib.

Visualization helps verify that the dataset contains handwritten digit images and allows the actual labels to be compared with the displayed images.

The images were displayed in grayscale because MNIST images contain grayscale pixel values.

9. Train-Test Split

The dataset was divided into training and testing sets.

The training data was used to teach the Machine Learning model, while the testing data was used to evaluate its performance on previously unseen images.

The dataset was divided using an **80:20 ratio**:

Dataset	Percentage
----------------	-------------------

Training Data	80%
---------------	-----

Testing Data	20%
--------------	-----

For the 10,000-record dataset:

Dataset	Records
----------------	----------------

Training Data	8,000
---------------	-------

Testing Data	2,000
--------------	-------

10. Machine Learning Model

The **K-Nearest Neighbors (KNN)** algorithm was used for handwritten digit classification.

KNN classifies a new image by comparing it with nearby training examples. The model identifies the closest examples and uses their labels to determine the predicted digit.

The model was configured using:

Number of Neighbors (K) = 3

The model was trained using the training dataset and then used to predict the labels of the test images.

11. Model Training

The KNN model was trained using the preprocessed training images and their corresponding digit labels.

The training process allows the model to learn the relationship between the pixel patterns and the corresponding digits.

After training, the model was used to classify the unseen test images.

12. Model Evaluation

The trained model was evaluated using the test dataset.

The main evaluation metric was **Accuracy**.

Accuracy measures the percentage of test images that were correctly classified.

The accuracy was calculated by comparing:

- Actual digit labels
- Predicted digit labels

Final Accuracy: 98 %

Enter the exact accuracy displayed by your Jupyter Notebook after running the model.

13. Classification Report

A classification report was generated to provide additional information about the model's performance for each digit from 0 to 9.

The report includes:

- Precision
- Recall
- F1-score
- Support

These metrics provide a more detailed evaluation than accuracy alone.

Classification Report:

Add the screenshot or output from your Jupyter Notebook here.

14. Confusion Matrix

A confusion matrix was created to analyze the predictions of the model.

It compares the actual digit with the predicted digit.

The diagonal values represent correctly classified images, while values outside the diagonal represent incorrect predictions.

The confusion matrix helps identify which digits are easier or more difficult for the model to recognize.

Confusion Matrix:

Add the confusion matrix screenshot from your notebook here.

15. Prediction Results

The trained model was tested on unseen images from the testing dataset.

Sample predictions were displayed by showing:

- The handwritten image
- Actual digit
- Predicted digit

This provides a visual demonstration of the model's ability to recognize handwritten digits.

Prediction Results:

Add screenshots of actual and predicted digits from your notebook here.

16. Results and Discussion

The MNIST Digit Recognition model successfully learned patterns from handwritten digit images and used these patterns to classify unseen images.

The KNN model was evaluated using test data, and its performance was measured using accuracy and other classification metrics.

The confusion matrix provides additional information about the digits that were correctly and incorrectly classified.

The final performance depends on the exact dataset split and model configuration used during training. Therefore, the accuracy reported in this project should be taken directly from the executed Jupyter Notebook.

17. Limitations

- KNN can require significant computational resources when working with many images.
 - Similar-looking handwritten digits can sometimes be difficult to classify.
 - The model's performance depends on the quality and distribution of the training data.
 - The project uses only pixel information without advanced image feature extraction.
 - Larger datasets may increase prediction time for KNN.
-

18. Future Improvements

Future improvements could include:

- Testing Logistic Regression.
- Testing Support Vector Machines (SVM).
- Testing Random Forest.
- Using Artificial Neural Networks.
- Using Convolutional Neural Networks (CNN).
- Increasing the amount of training data.
- Performing hyperparameter tuning.

- Comparing multiple Machine Learning models.
 - Building a web application for real-time handwritten digit recognition.
-

19. Conclusion

This project successfully developed a **Machine Learning-based MNIST Digit Recognition system** for classifying handwritten digits from **0 to 9**.

The images were pre-processed by separating the labels and pixel features and normalizing the pixel values. A **K-Nearest Neighbors (KNN)** model was trained using the processed data.

The model was evaluated using test data, accuracy, classification metrics, and a confusion matrix. The project demonstrates the complete Machine Learning workflow from **data preprocessing to model training and evaluation**.

Overall, the project shows how Machine Learning can be effectively used for handwritten digit image classification.

20. References

- MNIST Handwritten Digit Dataset
- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- K-Nearest Neighbors (KNN)
- Jupyter Notebook